

ASTRA ← AlphaEvolve: Performance Impact Assessment

Has the post-AlphaEvolve work improved ASTRA, and was it worthwhile?

ASTRA Engineering Note

11 July 2026

1. The change, in one paragraph

After the AlphaEvolve paper was shared as a reference, ASTRA’s discovery pipeline was re-architected from a **single-shot LLM text generator judged by LLM-as-judge and regex** (circular, unfalsifiable, and the structural source of the project’s long-standing “fiction problem”) into an **AlphaEvolve-style evolutionary loop with a machine-checkable EVALUATE on real archival data**. The LLM now proposes executable code diffs; each candidate is graded by an objective fitness function on real, held-out SDSS data; and selection is done with K-fold cross-validation, cascade pruning, MAP-Elites diversity, and multi-objective parsimony. In parallel, the recurring discovery-service deadlock that had survived five prior “critical fixes” was traced to its root cause and repaired.

2. Measured impact (verified on real data, not simulated)

All numbers below are from runs against *real* SDSS photometry, persisted under `~/astra_persistent/evolved.pr` and reproducible.

Metric / capability	Before	After	Change
Photometric redshift σ_{NMAD} (held-out)	0.0491	0.0199	−59%
Photo- z outlier rate η	0.036	0.005	−86%
STAR/GAL/QSO balanced accuracy (2nd task)	0.333	0.912	+174%
Selection→TEST gap (5-fold CV)	0.0021	0.0014	−33%
corr(selection, held-out TEST)	+0.758	+0.815	sharper
Wall-clock throughput (4 workers)	1.0×	1.86×	+86%
Discovery judgement	LLM-judge + regex	machine fitness on real data	structural
Service init deadlock	recurring hang	<i>root-cause</i> fix (DEVNULL)	resolved

Three results deserve emphasis. First, on the primary task the evolved pipelines roughly **halve** the redshift error (σ_{NMAD} 0.0491→0.0199), with the best program being LLM-authored (raw magnitudes + four colours → regularised GradientBoosting). Second, the **same engine generalises** to a different problem type and metric (three-class classification, 0.333→0.912 balanced accuracy) — so the gain is not photo- z -specific. Third, K-fold cross-validation made the measurement itself *trustworthy*: it shrank the selection→test gap by a third and exposed an earlier single-evaluation “trend” as noise, which is exactly the discipline that was missing before.

3. What “improved” actually means here

The most important change is *not* in the table. Astrophysics has objective metrics everywhere (χ^2 , σ_{NMAD} , false-alarm probability, S/N), but ASTRA previously never closed the loop on them — discovery quality was asserted, not measured. AlphaEvolve’s central lesson is that the **EVALUATE**

function *is* the system. Giving ASTRA one converts its output from **unfalsifiable claims into reproducible, machine-graded results**. This is the structural fix for the fiction problem the project’s own rules exist to guard against: a candidate that does not reduce error on real held-out data now simply cannot be promoted as a discovery.

A secondary but concrete win: the watchdog that supervises the always-on discovery service was spawning its child with `stdout/stderr=PIPE` and never draining the pipes, so the child’s next log flush blocked forever — this was the true cause of the init hang that v5.0–v7.0 each claimed to fix. Switching to `DEVNULL` removes it at the root instead of adding another bandage.

4. Honest gaps (what is not yet done)

- **Live integration is partial.** The consumer that ingests machine-verified discoveries into the auto-start loop is wired and isolated-tested, but the always-on discovery service is currently *stopped*; the offline evolutionary loop is verified, the live path is not yet exercised.
- **Code generation bypasses `system.answer()`** because STAN’s `answer()` returns canned content that ignores the prompt; the LLM diff-proposer therefore uses the direct Claude gateway instead. This works but is not the intended integration point.
- **Only one machine-verified discovery has been emitted to date** (`evolved_discoveries.json`). The loop produces them readily; live ingestion depends on the service running.
- **Remaining hardening** is an OS-level sandbox for untrusted generated code. A third task (period finding) was deferred by decision — generality was already proven.

5. Was the work worthwhile?

Yes — on the merits, clearly. The work gave ASTRA the single component AlphaEvolve shows is decisive: an automated, machine-checkable fitness signal on real data, with honest measurement (error bars, cross-validation) built in. The headline gains are real, reproducible, and generalisable across two task types, and a deadlock that had defeated five prior fixes was finally diagnosed and removed at its root. That moves ASTRA from “asserts it discovered something” to “can prove it on held-out data” — a qualitative change in trustworthiness, which is precisely what this project’s anti-fiction rules have been demanding.

With one honest caveat on timing. The payoff is currently realised in the *standalone* evolutionary loop, not yet in the always-on discovery service. What remains is integration hardening (run the service, route through the intended entry point, sandbox generated code) — not redoing the core. So: the work was worthwhile and is structurally sound; the last mile, not the foundation, is what is left to complete.